I 320D · Data Science for Biomedical Informatics · Spring 2026

Step 3: Missing Data & Imputation

Expanded Reference · Challenge Questions with Full Answers

University of Texas at Austin · School of Information

What this document contains

This cheatsheet expands Step 3 of the Project 4 notebook into a complete standalone reference covering:

- Full step-by-step imputation instructions: forward-fill, median fallback, leakage prevention, and verification
- Q13: Staleness thresholds — feature-by-feature decay tables, full implementation with hours_since_measured column, and grid-search code
- Q14: Forward-fill vs rolling-window mean — experimental comparison code, expected AUROC results, and interpretation framework
- Q15: Missingness indicator features — which features earn flags, SHAP validation, and clinical interpretation of indicator importance
- Q16: ER-admission cold-start problem — three imputation strategies compared, clinical rationale for each, and production handling
- Self-assessment checklist with exam-weight markers

PART 1 — Step-by-Step Imputation Instructions

1. Create missingness flags BEFORE any imputation

Binary flags must be created from the raw data before any filling occurs. Once you forward-fill, you can no longer distinguish real from imputed values.

```
# Features with >50% missingness earn a flag (informative missingness)
FEATURE_COLS = [c for c in df.columns
                if c not in ('PatientID','Hour','SepsisLabel')]

miss_rate = df[FEATURE_COLS].isnull().mean()
FLAG_FEATURES = miss_rate[miss_rate > 0.50].index.tolist()
print(f'Features receiving missingness flags: {len(FLAG_FEATURES)}')
print(FLAG_FEATURES)

# Create flags on the ORIGINAL df (before any imputation)
for col in FLAG_FEATURES:
    df[f'{col}_observed'] = df[col].notna().astype(int)

print('Flags created:', [c for c in df.columns if c.endswith('_observed')])
```

2. Apply patient-level forward-fill

Forward-fill within each patient's sorted timeline. Never apply ffill across patient boundaries.

```
def impute_patient(grp, train_medians):
    """Forward-fill within patient, then fill remaining NaNs with training
medians."""
    grp    = grp.sort_values('Hour').copy()
    filled = grp[FEATURE_COLS].ffill()       # carry last observed value
forward
    filled = filled.fillna(train_medians)    # fill leading NaNs with median
    grp[FEATURE_COLS] = filled
    return grp

# CRITICAL: compute medians on TRAINING patients only
# (assumes df_train is defined from your patient-level split)
TRAIN_MEDIANS = df_train[FEATURE_COLS].median()

df_train_imp = (df_train.groupby('PatientID', group_keys=False)
                        .apply(lambda g: impute_patient(g, TRAIN_MEDIANS)))
df_test_imp  = (df_test.groupby('PatientID', group_keys=False)
                       .apply(lambda g: impute_patient(g, TRAIN_MEDIANS)))

# Verify zero NaNs remain
assert df_train_imp[FEATURE_COLS].isnull().sum().sum() == 0, 'NaNs remain in
train'
assert df_test_imp[FEATURE_COLS].isnull().sum().sum()  == 0, 'NaNs remain in
test'
print('Imputation complete. Zero NaNs confirmed.')
```

3. Why train-only medians prevent leakage

```
WRONG workflow (leakage):                  CORRECT workflow:

1. Compute medians on ALL data             1. Split patients into train / test
```

```
  2. Impute using full-data medians          2. Compute medians on TRAIN only
  3. Split into train / test                 3. Impute train with train medians
                                             4. Impute test  with train medians


  Why wrong: test-set statistics (e.g.       Why correct: at deployment time, the
  higher sepsis patient medians) bleed        'test set' is future patients you
have
  into training imputation.                  never seen. You can only use
historical
                                             (training) data to derive imputation
stats.


  Inflation: typically +0.5 to +2.0          This simulates real deployment
faithfully.
  AUROC points on imbalanced datasets.
```

PART 2 — Challenge Questions: Full Answers

Q13 Staleness thresholds: when does a forward-filled value become clinically meaningless?

HINT *Forward-fill can propagate a value 20+ hours into the future. At what point does a propagated value become clinically meaningless? Design a 'staleness' threshold and implement it.*

✓ THE SHORT ANSWER

A forward-filled value becomes clinically stale when **more time has elapsed than the typical physiological half-life of that measurement.** For rapidly changing vital signs like lactate or SpO2 during active resuscitation, a 4-hour-old value may be meaningless. For slowly changing labs like creatinine or sodium, a 12-hour-old value may still be valid. The correct approach is a **feature-specific staleness threshold** rather than a single cutoff applied uniformly. Values older than their threshold should be replaced with the training median, and the *hours_since_measured* gap should be added as an explicit feature.

✓ PHYSIOLOGICAL BASIS FOR STALENESS THRESHOLDS

Feature	Physiological half-life / change rate	Recommended staleness threshold	Clinical rationale
Lactate	Changes 1–4 mmol/L per hour in active sepsis; clears in 2–6h with treatment	4 hours	Lactate clearance is the primary resuscitation endpoint; a 6h-old lactate gives no information about current perfusion status
SpO2	Can drop dramatically in minutes during respiratory failure	2 hours	Real-time vital; sensor dropout of >2h means clinical state is genuinely unknown
HR	Changes beat-to-beat; rolling trends shift over 1–2h	4 hours	Compensatory tachycardia resolves quickly with fluid; stale HR hides the response
SBP / MAP	Responds to vasopressors within minutes; drifts over hours	4 hours	MAP is the vasopressor titration target; a 4h-old MAP tells you nothing about current haemodynamics
Temp	Changes 0.5–1°C per hour in febrile response	8 hours	Temperature peaks and defervesces over hours; 8h-old value still reflects approximate status

Creatinine	Changes 0.1–0.3 mg/dL per day in acute injury	24 hours	AKI progresses slowly; a 12–24h-old creatinine is still clinically useful
WBC	Changes over hours during severe infection	12 hours	WBC trends are useful but not minute-to-minute; 12h is clinically reasonable
Sodium	Changes 1–2 mEq/L per day unless actively treated	24 hours	Dysnatraemia is corrected slowly; day-old sodium is still meaningful
Glucose	Can change 50–100 mg/dL per hour in critical illness	4 hours	Glucose is actively managed in ICU; hourly monitoring is standard; stale glucose is dangerous
pH / PaCO2	Changes rapidly during respiratory failure or resuscitation	4 hours	Acid-base status drives ventilator management; old ABG is unreliable

✓ COMPLETE IMPLEMENTATION

```
import numpy as np
import pandas as pd

# ── Feature-specific staleness thresholds (hours) ──────────────────────
STALENESS_THRESHOLDS = {
    'HR':        4,   'O2Sat':   2,   'Temp':     8,
    'SBP':       4,   'MAP':     4,   'DBP':      4,
    'Resp':      4,   'EtCO2':   2,
    'Lactate':   4,   'Glucose': 4,   'pH':       4,
    'PaCO2':     4,   'Creatinine': 24, 'BUN':    24,
    'Sodium':   24,   'Potassium': 12,  'WBC':    12,
    'Hgb':      12,   'Hct':     12,   'Platelets': 12,
    'Fibrinogen': 12, 'PTT':     12,   'TroponinI': 12,
}
DEFAULT_STALENESS = 8  # fallback for any feature not listed above

def impute_with_staleness(grp, train_medians, staleness_thresholds,
default=8):
    """
    Forward-fill within patient, but replace values older than the staleness
    threshold with the training median. Also creates hours_since_measured
columns.
    """
    grp = grp.sort_values('Hour').copy()

    for col in FEATURE_COLS:
        if col not in grp.columns:
            continue
        threshold = staleness_thresholds.get(col, default)
        series    = grp[col].copy()

        # Track hours since last real measurement
        hours_since = np.full(len(series), np.nan)
```

```
        last_obs_hour = None

        for i, (idx, val) in enumerate(series.items()):
            hour = grp.loc[idx, 'Hour']
            if pd.notna(val):
                last_obs_hour = hour
                hours_since[i] = 0
            elif last_obs_hour is not None:
                hours_since[i] = hour - last_obs_hour

        grp[f'{col}_hours_since'] = hours_since

        # Forward-fill
        filled = series.ffill()

        # Replace stale values (gap > threshold) with training median
        stale_mask = pd.Series(hours_since, index=series.index) > threshold
        filled[stale_mask] = train_medians.get(col, filled.median())

        # Fill remaining leading NaNs with training median
        filled = filled.fillna(train_medians.get(col, series.median()))

        # Fill hours_since NaN (leading — no prior measurement) with 999 flag
        grp[f'{col}_hours_since'] = grp[f'{col}_hours_since'].fillna(999)
        grp[col] = filled

    return grp

print('Applying staleness-aware imputation...')
df_train_stale = (df_train.groupby('PatientID', group_keys=False)
                          .apply(lambda g: impute_with_staleness(
                              g, TRAIN_MEDIANS, STALENESS_THRESHOLDS)))

# Verify
stale_replaced = (df_train_stale['Lactate_hours_since'] > 4).sum()
print(f'Lactate values replaced due to staleness: {stale_replaced:,}')
```

✓ GRID-SEARCH: FINDING OPTIMAL STALENESS THRESHOLDS

```
from sklearn.linear_model import LogisticRegression
from sklearn.metrics        import roc_auc_score
from sklearn.preprocessing  import StandardScaler

# Grid-search staleness threshold for Lactate (most clinically important)
thresholds_to_try = [2, 4, 6, 8, 12, 24, 9999]  # 9999 = no staleness limit
results = []

for t in thresholds_to_try:
    test_thresh = STALENESS_THRESHOLDS.copy()
    test_thresh['Lactate'] = t

    df_tr = (df_train.groupby('PatientID', group_keys=False)
                     .apply(lambda g: impute_with_staleness(
                         g, TRAIN_MEDIANS, test_thresh)))
    df_te = (df_test.groupby('PatientID', group_keys=False)
                    .apply(lambda g: impute_with_staleness(
                        g, TRAIN_MEDIANS, test_thresh)))
```

```
    feature_cols = [c for c in df_tr.columns
                    if c not in ('PatientID','Hour','SepsisLabel')]

    scaler  = StandardScaler()
    X_tr    = scaler.fit_transform(df_tr[feature_cols].fillna(0))
    X_te    = scaler.transform(df_te[feature_cols].fillna(0))
    y_tr    = df_tr['SepsisLabel'].values
    y_te    = df_te['SepsisLabel'].values

    lr = LogisticRegression(max_iter=500, class_weight='balanced',
                            C=0.1, random_state=42)
    lr.fit(X_tr, y_tr)
    auroc = roc_auc_score(y_te, lr.predict_proba(X_te)[:,1])
    results.append({'threshold': t, 'AUROC': auroc})
    print(f'  Lactate staleness {t:>5}h  ->  AUROC = {auroc:.4f}')

import pandas as pd
print(pd.DataFrame(results).to_string(index=False))
```

Key exam takeaway

The correct answer is not a single threshold — it is the principle of feature-specific thresholds based on physiological change rate. A student who proposes '8 hours for everything' has the right idea but misses the clinical nuance.

The hours_since_measured column is as important as the staleness correction itself. It gives the model explicit information about data recency and allows it to learn 'a creatinine I measured 20 hours ago is less trustworthy than one I measured 1 hour ago' — which is exactly how clinicians think about lab interpretation.

Expected AUROC finding: moderate improvement (0.5–1.5 AUROC points) from staleness correction, concentrated in features with short physiological half-lives (Lactate, Glucose, pH). Long-half-life features (Creatinine, Sodium) show negligible AUROC change.

Q14 Forward-fill vs rolling-window mean imputation: does the approach matter?

HINT *Instead of forward-fill, try mean imputation within a 6-hour rolling window. Compare AUROC between the two approaches. What does the difference tell you?*

✓ THE SHORT ANSWER

Rolling-window mean imputation fills each missing value with the average of the *surrounding* observations rather than the most recent one. For stable vital signs, the two approaches yield similar results. For rapidly changing features during active deterioration, forward-fill preserves the directional trajectory better — a rising Lactate carried forward for 2 hours is more clinically accurate than an average of recent and upcoming values. The AUROC comparison quantifies which effect dominates in your data.

✓ CONCEPTUAL DIAGRAM: HOW THE APPROACHES DIFFER

```
  Scenario: Lactate measured at hours 2 (1.2) and 8 (3.8) -- gap at hours
3,4,5,6,7

  True trajectory (unknown): 1.2  1.5  1.9  2.4  2.9  3.4  3.8
  Hour:                        2    3    4    5    6    7    8

  Forward-fill (LOCF):       1.2  1.2  1.2  1.2  1.2  1.2  3.8
    -> Flat line: misses the rising trajectory entirely
    -> But: directionally correct at hour 3 (still near 1.2, not yet 3.8)

  Rolling-window mean (6h):  1.2  2.1  2.5  2.5  2.5  2.8  3.8
  (average of values within +/- 3 hours of each missing point)
    -> Smoother interpolation, closer to true values in the middle
    -> But: USES FUTURE DATA (hour 8 value) to fill hour 3 -- data leakage!

  Linear interpolation:      1.2  1.65 2.1  2.55 3.0  3.45 3.8
    -> Closest to truth IF the gap is short and trajectory is monotone
    -> Also uses future data -- same leakage concern

  KEY INSIGHT: Only forward-fill is causal (uses only past data).
  Rolling mean and interpolation are anti-causal -- they peek at future values.
```

✓ THE LEAKAGE PROBLEM IN ROLLING-WINDOW MEAN

Anti-causal imputation is a subtle but serious error. When you compute the rolling mean over a window centred at hour t, you are averaging values from hours t-3 to t+3. The values at hours t+1, t+2, t+3 are **future values relative to hour t** — a patient at hour 3 does not know their hour 6 lab result. A model trained with anti-causal imputation learns patterns that cannot be reproduced at deployment time, where you only have past and current data. AUROC will be inflated compared to a causally-imputed model.

✓ COMPLETE EXPERIMENTAL COMPARISON CODE

```
import numpy as np
import pandas as pd
```

```
from sklearn.linear_model  import LogisticRegression
from sklearn.preprocessing import StandardScaler
from sklearn.metrics       import roc_auc_score

def impute_forward_fill(grp, train_medians):
    """Causal: uses only past data."""
    grp    = grp.sort_values('Hour').copy()
    filled = grp[FEATURE_COLS].ffill().fillna(train_medians)
    grp[FEATURE_COLS] = filled
    return grp

def impute_rolling_mean(grp, train_medians, window=6):
    """
    Anti-causal rolling mean: centres window at each missing point.
    Included for comparison; NOT recommended for production use.
    """
    grp    = grp.sort_values('Hour').copy()
    filled = grp[FEATURE_COLS].copy()
    for col in FEATURE_COLS:
        # Rolling mean using centred window (looks backward AND forward)
        roll_mean = (filled[col]
                     .fillna(method=None)  # keep NaN as NaN
                     .rolling(window=window, min_periods=1, center=True)
                     .mean())
        filled[col] = filled[col].fillna(roll_mean)    # fill NaN with local
mean
        filled[col] = filled[col].fillna(train_medians[col])  # fill
remaining
    grp[FEATURE_COLS] = filled
    return grp

def impute_rolling_mean_causal(grp, train_medians, window=6):
    """
    Causal rolling mean: uses only the past 'window' hours.
    Equivalent to forward-filling then smoothing backward.
    """
    grp    = grp.sort_values('Hour').copy()
    filled = grp[FEATURE_COLS].copy()
    for col in FEATURE_COLS:
        # Step 1: forward-fill so every gap is the last known value
        fwd = filled[col].ffill()
        # Step 2: take causal (trailing) rolling mean over the filled series
        smoothed = fwd.rolling(window=window, min_periods=1).mean()
        filled[col] = smoothed.fillna(train_medians[col])
    grp[FEATURE_COLS] = filled
    return grp

# ── Run all three imputation strategies and compare AUROC ─────────────────
strategies = {
    'Forward-fill (causal)':        impute_forward_fill,
    'Rolling mean centred (anti-causal)': impute_rolling_mean,
    'Rolling mean causal':           impute_rolling_mean_causal,
}

for name, impute_fn in strategies.items():
    df_tr = (df_train.groupby('PatientID', group_keys=False)
                     .apply(lambda g: impute_fn(g, TRAIN_MEDIANS)))
    df_te = (df_test.groupby('PatientID', group_keys=False)
                    .apply(lambda g: impute_fn(g, TRAIN_MEDIANS)))

    X_tr = df_tr[FEATURE_COLS].fillna(0).values
    y_tr = df_tr['SepsisLabel'].values
```

```
X_te = df_te[FEATURE_COLS].fillna(0).values
y_te = df_te['SepsisLabel'].values

sc   = StandardScaler()
X_tr = sc.fit_transform(X_tr)
X_te = sc.transform(X_te)

lr   = LogisticRegression(max_iter=500, class_weight='balanced',
                          C=0.1, random_state=42)
lr.fit(X_tr, y_tr)
auroc = roc_auc_score(y_te, lr.predict_proba(X_te)[:,1])
print(f'{name:<45}  AUROC = {auroc:.4f}')
```

✓ EXPECTED RESULTS AND INTERPRETATION

Strategy	Expected AUROC (relative)	Why	Recommended?
Forward-fill (causal)	Baseline (e.g. 0.850)	Uses only past data; no leakage; simple	YES — default choice
Rolling mean centred (anti-causal)	0.5 – 2.0 points higher than baseline	Inflated by future-data leakage; at deployment time performance will regress to baseline	NO — leakage makes this invalid
Rolling mean causal	Within 0.2 AUROC of forward-fill	Equivalent information; slightly smoother trajectory	YES — valid alternative, marginal benefit over ffill
Linear interpolation (causal)	Within 0.3 AUROC of forward-fill	Better approximates true trajectory but requires careful causal implementation	YES — with min_periods=1 trailing only

✓ WHAT THE AUROC DIFFERENCE TELLS YOU

If centred rolling mean AUROC >> forward-fill AUROC	If centred rolling mean AUROC ≈ forward-fill AUROC
Interpretation	The performance gap is largely an artefact of anti-causal imputation leaking future information into the prediction. The model is benefiting from knowing the future trajectory.
Action	Report both with a note explaining the leakage. Use forward-fill for all published results. The gap size estimates the maximum benefit you could get from better trajectory estimation (e.g. a process model for each patient's vital sign dynamics).

Real-world implication	Anti-causal AUROC is an upper bound on what could be achieved with a perfect trajectory model (which is impossible at deployment). Forward-fill AUROC is the deployable reality.

Key exam takeaway

The fundamental rule: any imputation method that uses future values (values from hours t+1, t+2, ...) is anti-causal and introduces leakage. At deployment time, you have never seen the future. Forward-fill is the gold standard precisely because it is causal — it uses only data available at time t.

The correct answer to 'what does the AUROC difference tell you?' is: if anti-causal imputation scores higher, it quantifies the performance ceiling if you had a perfect trajectory model, and it demonstrates that the imputation strategy itself can be a source of data leakage that inflates published benchmark results.

Q15 Missingness indicator features: which flags improve prediction and what does that imply?

HINT *Add a binary missingness indicator feature for each variable (1 = originally observed, 0 = imputed). Retrain your model. Do any missingness indicators improve prediction? What does that imply clinically?*

✓ THE SHORT ANSWER

Some missingness indicators will improve prediction substantially. When a missingness indicator for feature X has a high positive model coefficient or high SHAP value, it means the **fact that X was ordered** is more predictive of sepsis than the **value of X itself.** This is the empirical signature of informative missingness — the clinician's ordering decision encodes their suspicion. The most important finding to look for: if *Lactate_observed* or *EtCO2_observed* have higher SHAP importance than their corresponding value features, the model has learned that clinical ordering behaviour is a powerful risk signal.

✓ COMPLETE CODE: FLAGS + SHAP VALIDATION

```
import numpy as np
import pandas as pd
from sklearn.linear_model  import LogisticRegression
from sklearn.preprocessing import StandardScaler
from sklearn.metrics       import roc_auc_score

# ── Step 1: Identify flag candidates ──────────────────────────────────
# Use features with > 40% missingness -- lower threshold than the 50% rule
# to capture borderline informative features
FEATURE_COLS_RAW = [c for c in df.columns
                    if c not in ('PatientID','Hour','SepsisLabel')]
miss_rate = df[FEATURE_COLS_RAW].isnull().mean()
FLAG_CANDIDATES = miss_rate[miss_rate > 0.40].index.tolist()
print(f'Flag candidates ({len(FLAG_CANDIDATES)}): {FLAG_CANDIDATES}')

# ── Step 2: Create flags on the ORIGINAL (pre-imputation) DataFrame ────
for col in FLAG_CANDIDATES:
    df[f'{col}_observed'] = df[col].notna().astype(int)

FLAG_COLS = [f'{c}_observed' for c in FLAG_CANDIDATES]

# ── Step 3: Impute as normal (flags already created) ──────────────────
# ... (run impute_patient as in notebook Step 3)

# ── Step 4: Train two models -- with and without flags ────────────────
ALL_FEATURE_COLS = FEATURE_COLS_RAW + FLAG_COLS

for label, feature_set in [('Without flags', FEATURE_COLS_RAW),
                           ('With flags',    ALL_FEATURE_COLS)]:
    X_tr = df_train[feature_set].fillna(0).values
    y_tr = df_train['SepsisLabel'].values
    X_te = df_test[feature_set].fillna(0).values
    y_te = df_test['SepsisLabel'].values

    sc   = StandardScaler()
```

```
    X_tr = sc.fit_transform(X_tr)
    X_te = sc.transform(X_te)

    lr   = LogisticRegression(max_iter=1000, class_weight='balanced',
                              C=0.1, random_state=42)
    lr.fit(X_tr, y_tr)
    auroc = roc_auc_score(y_te, lr.predict_proba(X_te)[:,1])
    print(f'{label:<25}  AUROC = {auroc:.4f}')

# ── Step 5: Identify which flags have the highest LR coefficients ──────
# Train the 'with flags' model and inspect coefficients
sc2  = StandardScaler()
X_tr2 = sc2.fit_transform(df_train[ALL_FEATURE_COLS].fillna(0))
lr2   = LogisticRegression(max_iter=1000, class_weight='balanced',
                           C=0.1, random_state=42)
lr2.fit(X_tr2, df_train['SepsisLabel'].values)

coef_df = pd.DataFrame({
    'feature': ALL_FEATURE_COLS,
    'coef':    lr2.coef_[0]
}).sort_values('coef', ascending=False)

print('\nTop 10 positive coefficients:')
print(coef_df.head(10).to_string(index=False))

print('\nFlag-only coefficients (sorted by absolute value):')
flag_coefs = coef_df[coef_df['feature'].str.endswith('_observed')]
print(flag_coefs.sort_values('coef', ascending=False).to_string(index=False))

# ── Step 6: SHAP validation ──────────────────────────────────────────
try:
    import shap
    X_bg    = shap.sample(X_tr2, 100, random_state=42)
    X_samp  = sc2.transform(df_test[ALL_FEATURE_COLS].fillna(0))[:500]
    explainer    = shap.LinearExplainer(lr2, X_bg)
    shap_values  = explainer.shap_values(X_samp)
    mean_abs_shap = np.abs(shap_values).mean(axis=0)
    shap_df = pd.DataFrame({'feature': ALL_FEATURE_COLS,
                            'mean_abs_shap': mean_abs_shap}
                           ).sort_values('mean_abs_shap', ascending=False)
    print('\nTop 15 features by mean |SHAP|:')
    print(shap_df.head(15).to_string(index=False))
    print('\nFlag features only, ranked by SHAP:')

print(shap_df[shap_df.feature.str.endswith('_observed')].head(10).to_string(index=Fal
se))
except ImportError:
    print('shap not installed -- run: pip install shap')
```

✓ INTERPRETING THE FINDINGS: A DECISION MATRIX

What you observe	What it means	Clinical implication	Action
Lactate_observed has high positive coefficient / SHAP	The model has learned: 'when a nurse orders lactate, the patient is probably sicker than average, because	Clinical ordering behaviour is itself a risk signal. Clinicians are implicitly triaging sepsis risk before the	Retain the flag. Report it prominently — it shows the model has learned

	lactate is ordered when perfusion looks poor'	model sees any numbers.	implicit clinical judgment.
EtCO2_observed has high positive coefficient / SHAP	Intubated patients have higher sepsis risk than non-intubated patients in this cohort	Intubation status is a severity marker. The model correctly identifies this even without an explicit 'intubated' column.	Retain the flag. Consider adding it to the clinical summary output ('Patient is intubated — higher sepsis risk baseline').
Lactate_observed importance > Lactate value importance	The model relies more on whether lactate was checked than on the actual lactate level	Extremely interesting finding: clinical ordering patterns predict sepsis better than the biomarker itself. This may reflect that delayed ordering is itself a risk factor.	Report as a key finding. Investigate whether late ordering (delayed lactate check) predicts worse outcomes — this is a patient safety question.
AUROC improves by > 0.01 with flags added	Missingness is not random in this dataset — it contains genuine clinical signal	A model without flags is leaving predictive information on the table. The flags should be included in the final feature set.	Include flags in production model. Document the clinical meaning of each important flag in model cards.
AUROC improves by < 0.005 with flags added	Missingness is approximately random in this dataset, or the value features already capture the information	The features that are missing are missing for random reasons (sensor dropout, charting lag) rather than clinical decisions	Flags optional — they add complexity without benefit. Consider dropping flags for simplicity.

✓ EXPECTED FINDINGS FOR THE PHYSIONET 2019 DATASET

Based on the clinical structure of the dataset, the following flags are most likely to show high importance:

Lactate_observed: High importance expected. Lactate is ordered when perfusion is clinically suspected to be inadequate. Its presence predates the formal sepsis label because it reflects clinical concern. If Lactate_observed has a SHAP value comparable to or exceeding Lactate itself, you have demonstrated empirically that clinical ordering behaviour is a leading indicator of sepsis.

EtCO2_observed: High importance expected. Intubation is a severity marker. This flag encodes respiratory failure severity directly.

TroponinI_observed: Moderate importance. Ordered when cardiac dysfunction is suspected — septic cardiomyopathy is common in severe sepsis. A positive TroponinI_observed flag indicates the team suspected cardiac involvement.

Fibrinogen_observed: Moderate importance. Ordered when DIC (disseminated intravascular coagulation) is suspected — a severe sepsis complication. High Fibrinogen_observed may precede the lab confirmation of DIC.

HR_observed, Temp_observed: Low importance expected. These are monitored continuously; their missingness reflects sensor/charting lag, not clinical decisions. No informative signal.

Key exam takeaway

The question 'do missingness indicators improve prediction?' has a three-part answer: (1) empirically yes for high-missingness clinically ordered features; (2) the improvement quantifies how much information is in the ordering pattern vs the value; and (3) if indicator importance exceeds value importance, this is a publishable finding about clinical workflow as a risk signal.

The deep insight: in clinical data, the act of measurement is an act of clinical judgment. A model that ignores the measurement pattern is ignoring the implicit risk stratification performed by the clinical team before any algorithm ran.

Q16 The ER-admission cold-start problem: imputing patients with no prior measurements

HINT *Patients admitted to the ICU directly from the ER may have no prior measurements for the first few hours. How does your imputation strategy behave for these patients at hour 0, and how should it be handled?*

✓ THE SHORT ANSWER

ER-admission patients have a **cold-start problem**: at ICU admission hour 0, all feature columns are NaN because no measurements have been taken yet. Forward-fill produces nothing (no prior value to carry forward). The default fallback — training-set median — treats this patient as 'average', which is clinically incorrect. ER patients are admitted to the ICU precisely because they are *not* average. The correct approach is a hierarchy of three strategies applied in order: (1) use the first available measurement back-filled to hour 0, (2) use ER-patient-specific medians if available, or (3) use the overall training median with a cold-start flag.

✓ THE COLD-START PROBLEM ILLUSTRATED

```
ER-direct admission patient timeline:

Hour:    0     1     2     3     4     5     6     7     8
HR:    NaN   NaN   NaN   NaN    92   NaN   NaN    98   NaN
Lact:  NaN   NaN   NaN   NaN   NaN   NaN   3.2   NaN   NaN
Creat: NaN   NaN   NaN   NaN   NaN   NaN   NaN   NaN   1.8

Forward-fill result (hours 0-3):
HR:    med   med   med   med    92    92    92    98    98
Lact:  med   med   med   med   med   med   3.2   3.2   3.2
Creat: med   med   med   med   med   med   med   med   1.8

Problem: hours 0-3 are entirely synthetic (population median).
The model treats this critically ill ER patient as 'average'
during the most important hours of their admission.

Better strategy: back-fill first available measurement to hour 0
HR:     92    92    92    92    92    92    92    98    98
Lact:  3.2   3.2   3.2   3.2   3.2   3.2   3.2   3.2   3.2
Creat: 1.8   1.8   1.8   1.8   1.8   1.8   1.8   1.8   1.8

This propagates the FIRST REAL MEASUREMENT backward -- much more
clinically defensible for ER admissions.
```

✓ THREE STRATEGIES COMPARED

~

STRATEGY A: Population median (current default)

Implementation: fill all leading NaNs with TRAIN_MEDIANS. Advantage: simple, already implemented. Disadvantage: treats every ER patient as having 'normal' vitals for the first few hours — the model cannot distinguish a stable elective admission from a critically ill ER patient at hour 0. Cold-start hours become uninformative dead zones in the time series.

✓ **STRATEGY B: Back-fill from first available measurement (recommended for vitals)**
Implementation: for each patient, find the first non-NaN value for each feature and back-fill it to all prior NaN hours. This assumes the first measured value is representative of the patient's state at admission — clinically reasonable for vitals (a patient with HR 110 at first measurement probably had HR ~110 at ICU admission). For labs, back-fill is more aggressive: a creatinine of 3.5 at hour 6 does not necessarily mean it was 3.5 at hour 0 (AKI may still be developing). Apply back-fill to vitals; use median fallback for labs.

✓ **STRATEGY C: ER-patient-specific medians + cold-start flag (recommended for labs)**
Implementation: separate the training patients into ER-admitted vs ward-admitted groups. Compute medians for ER patients separately. Use ER-specific medians for ER-admission imputation. Also add a binary is_er_admission flag (or equivalently, a cold_start flag = 1 for hours before the first measurement). The cold-start flag tells the model to treat feature values in those hours with less confidence. This is the most principled approach.

✓ COMPLETE CODE: ALL THREE STRATEGIES

```
import numpy as np
import pandas as pd

def detect_er_admissions(df, threshold_hours=4):
    """
    Identify ER-direct admissions: patients where the first measurement
    for any vital sign appears after hour 0 (no hour-0 data for vitals).
    threshold_hours: patients with first vital measurement after this hour
    are classified as ER admissions.
    """
    VITALS = ['HR','O2Sat','SBP','MAP','Resp']
    er_patients = []
    for pid, grp in df.groupby('PatientID'):
        grp = grp.sort_values('Hour')
        first_vital_hour = grp[VITALS].dropna(how='all')['Hour'].min() \
            if 'Hour' in grp[VITALS].dropna(how='all').columns \
            else grp[grp[VITALS].notna().any(axis=1)]['Hour'].min()
        if pd.isna(first_vital_hour) or first_vital_hour > threshold_hours:
            er_patients.append(pid)
    return set(er_patients)

er_pids = detect_er_admissions(df_train)
print(f'ER-admission patients in training set: {len(er_pids):,}
({100*len(er_pids)/df_train.PatientID.nunique():.1f}%)')

# ── Strategy B: back-fill first measurement to admission for vitals ──────
VITALS = ['HR','O2Sat','Temp','SBP','MAP','DBP','Resp','EtCO2']

def impute_with_backfill_vitals(grp, train_medians, er_pids):
    """Apply back-fill to vitals for ER admissions; forward-fill for all
others."""
    grp    = grp.sort_values('Hour').copy()
    pid    = grp['PatientID'].iloc[0]
    is_er  = pid in er_pids

    for col in FEATURE_COLS:
        series = grp[col].copy()
```

```
        # Forward-fill first (handles gaps mid-stay)
        filled = series.ffill()
        # For ER admissions: also back-fill first observation to hour 0 for
vitals
        if is_er and col in VITALS:
            filled = filled.bfill()   # propagates first value backward to
hour 0
        # Final fallback: training median
        filled = filled.fillna(train_medians[col])
        grp[col] = filled

    # Add cold-start flag: 1 for hours before first vital measurement
    first_obs_hour = grp[VITALS].notna().any(axis=1)
    first_obs_idx  = first_obs_hour[first_obs_hour].index[0] \
        if first_obs_hour.any() else grp.index[-1]
    grp['cold_start'] = 0
    grp.loc[:first_obs_idx, 'cold_start'] = 1

    return grp

print('Applying back-fill imputation for ER admissions...')
df_train_v2 = (df_train.groupby('PatientID', group_keys=False)
                        .apply(lambda g: impute_with_backfill_vitals(
                            g, TRAIN_MEDIANS, er_pids)))

# ── Strategy C: ER-specific medians ──────────────────────────────────────
er_mask    = df_train['PatientID'].isin(er_pids)
ER_MEDIANS = df_train[er_mask][FEATURE_COLS].median()
print('ER-patient medians (first 5 features):')
print(ER_MEDIANS.head().round(2))
print('Training-set overall medians (first 5):')
print(TRAIN_MEDIANS.head().round(2))
print('Difference (ER - Overall):')
print((ER_MEDIANS - TRAIN_MEDIANS).head().round(2))
```

✓ EXPECTED FINDINGS: ER vs OVERALL PATIENT MEDIANS

Feature	Expected ER median (approx)	Expected overall median (approx)	Direction of difference	Clinical explanation
HR	100 – 110 bpm	80 – 90 bpm	ER higher by 10–20 bpm	ER patients arrive in physiological distress; tachycardia is common at ED presentation
SBP	100 – 115 mmHg	115 – 125 mmHg	ER lower by 5–15 mmHg	Hypotension is a frequent ER presentation trigger for ICU admission
Lactate	3.0 – 5.0 mmol/L	1.5 – 2.5 mmol/L	ER higher by 1–3 mmol/L	Elevated lactate is a key triage criterion for ICU admission from the ER
Resp	22 – 28 breaths/min	16 – 20 breaths/min	ER higher by 4–8 RR	Tachypnoea is a SIRS criterion and common ER presentation trigger

Temp	38.0 – 38.8°C	37.0 – 37.5°C	ER higher by 0.5–1.5°C	Fever is a common ER presentation trigger; directly raises clinical suspicion

Key finding: ER patients are systematically sicker at baseline than the average training patient. Using the overall training median for ER-admission imputation sets the model's baseline at 'average sick patient' when it should be 'critically ill ER patient'. This will cause the model to underestimate risk for ER admissions at hour 0 — the exact period when early detection is most impactful.

✓ PRODUCTION HANDLING: A DECISION TREE

```
At ICU admission (hour 0 of model run):

Is this an ER-direct admission?
(no prior measurements in first 4 hours)
     |
    YES                              NO (ward, PACU, step-down transfer)
     |                                         |
Are any vitals available              Use standard forward-fill + median
from the ER chart?                   imputation from the first hour
     |
    YES                              NO
     |                                |
Use ER vitals as hour-0          Set cold_start = 1
feature values (back-fill)       Use ER-specific medians
cold_start = 0                   Flag output with lower confidence
                                 'Insufficient data -- prediction
                                  may be unreliable'

In all cases:
-> Include cold_start binary flag as a model feature
-> Track hours_since_first_measurement for the first 6 ICU hours
-> Do not suppress the alarm but flag with reduced confidence
   until at least 3 hours of real measurements are available
```

Key exam takeaway

The cold-start problem is not just a technical imputation challenge — it is a patient safety issue. ER admissions are disproportionately likely to be sepsis patients (they arrive sick), yet the default imputation strategy makes them look average. A model that outputs low sepsis risk at hour 0 for an ER admission is potentially dangerous.

The three-part answer: (1) back-fill vitals from first available measurement for ER patients; (2) use ER-patient-specific medians for labs; (3) add a cold_start binary flag so the model can learn to weight ER-admission hours differently. Report cold_start hours separately in model evaluation.

For a production deployment, add a human-readable output tag: 'Prediction based on fewer than 3 hours of data — validate with clinical assessment.' This is the responsible AI disclosure practice for a model used in a clinical decision support context.

Self-Assessment Checklist — Step 3

Check each item after completing Step 3. Items marked ★ are likely exam questions.

Imputation Fundamentals

- ☐ ★ I can explain why forward-fill is the clinically motivated default (LOCF principle)
- ☐ I can write the impute_patient function from memory (ffill then fillna(medians))
- ☐ ★ I know the correct order: split patients FIRST, then compute medians on train only
- ☐ I can run the zero-NaN assertion and explain what to investigate if it fails

Q13 — Staleness Thresholds

- ☐ ★ I can name 4 features with short staleness thresholds (< 6h) and explain the physiological basis for each
- ☐ I can name 3 features with long staleness thresholds (> 12h) and why
- ☐ ★ I can implement the hours_since_measured column and explain its value as a model feature
- ☐ I can write the grid-search code for optimising a single feature's staleness threshold
- ☐ I know the expected AUROC impact range: 0.5 – 1.5 points for rapidly-changing features

Q14 — Imputation Strategy Comparison

- ☐ ★ I can explain what makes rolling-window mean anti-causal and why this produces leakage
- ☐ I can draw the diagram showing forward-fill vs centred rolling mean on a 6-hour gap
- ☐ ★ I know that higher AUROC for anti-causal imputation is an artefact, not a real improvement
- ☐ I can explain what the AUROC gap between causal and anti-causal imputation represents (upper bound from trajectory knowledge)
- ☐ I can implement causal rolling mean using ffill() then trailing rolling().mean()

Q15 — Missingness Indicators

- ☐ ★ I know the 5 features most likely to produce informative missingness indicators in this dataset
- ☐ I can explain what it means clinically when Lactate_observed SHAP > Lactate SHAP ★
- ☐ I can write the two-model AUROC comparison code (with and without flags)
- ☐ ★ I know the rule: flags are informative when missingness reflects a clinical decision, not a sensor event
- ☐ I can interpret a positive coefficient on a missingness flag in clinical terms

Q16 — ER Admission Cold-Start

- ☐ ★ I can describe the cold-start problem and why the default strategy is clinically dangerous
- ☐ I can explain why ER patients have higher baseline vital signs than the training median ★
- ☐ I know all three strategies (median / back-fill / ER-specific medians) and when to use each
- ☐ ★ I can argue for adding a cold_start binary flag and explain what the model learns from it
- ☐ I know the responsible production output for cold-start hours: flag with reduced confidence

How to use this cheatsheet responsibly

Step 3 questions test both technical implementation skills and clinical reasoning. Items marked ★ should be answered without reference to this document in exam conditions. The code in this

cheatsheet is production-quality and can be used directly in your notebook submissions — attribution to this course material is sufficient.